

STEP Cleaning Project Architecture

This document explains the shape of the project: the folders, entrypoints, data flow, and how the modules relate to each other.

The project has one main purpose:

Given a STEP/STP CAD file, predict which CAD faces are likely to stay dirty after cleaning.

1. Architecture At A Glance

```
Root launchers
|
+-- train_hotspot_classifier.py
|   -> stepclean/ml/classifier.py
|   -> stepclean/ml/training_data.py
|   -> data/training/cleaning_retention_wf05/*.csv
|   -> cache/lightgbm_hotspot_model.txt
|
+-- desktop_ui.py
|   -> stepclean/app/desktop.py
|   -> stepclean/prediction/hotspots.py
|   -> cache/lightgbm_hotspot_model.txt
|
+-- analyze_cleaning_particle_retention.py
|   -> tools/data/cleaning_retention.py
|
+-- analyze_hotspot_feature_diagnostics.py
|   -> tools/diagnostics/hotspot_features.py
```

The root files are only launchers. They exist so commands stay simple. The real implementation is inside `stepclean/`, `features/`, and `tools/`.

2. Main Folders

```

stepclean/
  app/          desktop user interface
  ml/           model training and training-data preparation
  prediction/   model inference for new STEP files

features/
  cleaning_simulation/ STEP mesh conversion and cleaning simulation
  edge_classification/ CAD edge convexity and concavity
  hole_finding/       face form detection and inward cylinder detection

tools/
  data/          training CSV generation
  diagnostics/   feature diagnostics and model experiments

data/
  training/      committed training CSVs used by the model

cache/
  generated model and prediction outputs

docs/
  project documentation PDFs and Markdown sources

```

3. Dependency Direction

The project is easiest to understand if you think in layers.

```

UI layer
  stepclean/app/desktop.py
  |
  v
Prediction layer
  stepclean/prediction/hotspots.py
  |
  v
Domain feature layer
  features/cleaning_simulation/
  features/edge_classification/
  features/hole_finding/
  |
  v
Model artifact and data
  cache/lightgbm_hotspot_model.txt
  data/training/cleaning_retention_wf05/

```

Training has a separate path:

```

Training launcher
  train_hotspot_classifier.py
  |
  v
Training logic
  stepclean/ml/classifier.py
  |
  v
Training-data builder
  stepclean/ml/training_data.py
  |
  v
Training CSVs
  data/training/cleaning_retention_wf05/

```

The UI does not train the model. It loads the already trained model from cache/.

4. Runtime Prediction Flow

This is what happens when a user opens a STEP file and clicks prediction in the UI.

```
desktop_ui.py
-> stepclean/app/desktop.py
-> predict_step_hotspots(...)
-> read STEP file
-> build surface mesh
-> run cleaning simulation
-> detect face forms
-> classify CAD edges
-> build 10 production features
-> load LightGBM model
-> return dirty probabilities
-> color predicted dirty faces in pink
```

Main runtime files:

- desktop_ui.py: tiny launcher.
- stepclean/app/desktop.py: Tkinter/OpenCascade interface.
- stepclean/prediction/hotspots.py: builds prediction features and calls the model.
- cache/lightgbm_hotspot_model.txt: trained LightGBM model.

5. Training Flow

This is what happens when the model is trained.

```
train_hotspot_classifier.py
-> stepclean/ml/classifier.py
-> build_training_frame(...)
-> read face_particle_retention.csv
-> read dirty_surface_neighbor_context.csv
-> read form_particle_retention.csv
-> aggregate neighbor and boundary context
-> keep the 10 production features
-> create target stayed dirty
-> grouped cross-validation by STEP object
-> train final LightGBM classifier
-> write cache/lightgbm_hotspot_model.txt
```

Main training files:

- train_hotspot_classifier.py: tiny launcher.
- stepclean/ml/classifier.py: LightGBM training, evaluation, prediction CSV writing.
- stepclean/ml/training_data.py: builds the training feature table.
- data/training/cleaning_retention_wf05/: source CSVs.

6. Data Contracts

The training CSVs are source data. They contain more columns than the model uses.

The model input is the contract that must stay aligned between training and prediction.

```
stepclean/ml/training_data.py    FEATURE_COLUMNS
stepclean/prediction/hotspots.py MODEL_FEATURE_COLUMNS
```

These two lists must match.

Current production model input:

```
area_weighted_exposure
sample_count
area_weighted_cleaning_dose
area_weighted_water_dose
neighbor_area_weighted_cleaning_dose_mean
area_weighted_poor_drainage
area_weighted_hiddenness
neighbor_area_weighted_hotspot_score_max
neighbor_area_weighted_cleaning_dose_min
neighbor_area_weighted_hotspot_score_mean
```

Current model size:

```
10 features
CV accuracy:    0.8977
Holdout accuracy: 0.8866
Threshold:      0.55
```

7. Data Folder

```
data/training/cleaning_retention_wf05/
face_particle_retention.csv
dirty_surface_neighbor_context.csv
form_particle_retention.csv
```

Responsibilities:

- face_particle_retention.csv: one row per face, including target information.
- dirty_surface_neighbor_context.csv: neighbor, boundary, and local-context information.
- form_particle_retention.csv: face-form summary data used by experiments and available for future feature changes.

8. Cache Folder

cache/ is generated output, not source architecture.

Current expected files:

```
cache/lightgbm_hotspot_model.txt
cache/hotspot_dirty_classifier_predictions.csv
cache/hotspot_dirty_classifier_holdout_predictions.csv
cache/hotspot_dirty_classifier_feature_importance.csv
```

The UI needs cache/lightgbm_hotspot_model.txt. The prediction and importance CSVs are for inspection.

9. Domain Feature Modules

The features/ package contains reusable CAD and cleaning logic.

features/cleaning_simulation/

Purpose:

- convert STEP shapes into sampled surface meshes
- simulate water exposure and cleaning dose
- estimate retained dust/particle behavior

Important files:

- mesh.py: STEP to surface mesh conversion.
- simulation.py: cleaning simulation.
- models.py: data structures.
- math_utils.py: vector math helpers.
- export.py: JSON/HTML export helpers.

features/edge_classification/

Purpose:

- classify CAD edges as convex, concave, or neutral
- provide edge context used by diagnostics and feature generation

Important file:

- classifier.py

features/hole_finding/

Purpose:

- detect face forms
- detect inward-cylinder candidates

Important files:

- face_forms.py
- inward_cylinder.py

10. Tooling Modules

Tools are command-line helpers. They are not part of normal UI prediction.

```
tools/data/cleaning_retention.py
```

Generates training CSVs by running cleaning analysis over STEP files.

```
tools/diagnostics/hotspot_features.py
```

Runs feature diagnostics such as PCA, mutual information, permutation importance, and ablation experiments.

11. Entrypoints

```
desktop_ui.py
```

Starts the desktop application.

```
train_hotspot_classifier.py
```

Trains the production classifier.

```
analyze_cleaning_particle_retention.py
```

Generates training CSVs.

```
analyze_hotspot_feature_diagnostics.py
```

Runs model/feature diagnostics.

12. Environment Split

The project uses two Python environments.

```
occwl-env
  CAD, OpenCascade, STEP reading, UI, tests

.venv311
  pandas, scikit-learn, LightGBM, diagnostics, training
```

Typical commands:

```
.\occwl-env\python.exe desktop_ui.py
.\occwl-env\python.exe -m unittest discover -s tests

.\.venv311\Scripts\python.exe train_hotspot_classifier.py
.\.venv311\Scripts\python.exe analyze_hotspot_feature_diagnostics.py
```

13. Change Map

If you change model features:

1. Update FEATURE COLUMNS in stepclean/ml/training_data.py.
2. Update MODEL_FEATURE COLUMNS in stepclean/prediction/hotspots.py.
3. Retrain with train_hotspot_classifier.py.
4. Update UI accuracy constants in stepclean/app/desktop.py.
5. Regenerate docs if the architecture or feature list changed.

If you change cleaning physics:

```
features/cleaning_simulation/
```

If you change CAD shape or edge analysis:

```
features/edge_classification/
features/hole_finding/
```

If you change the interface:

```
stepclean/app/desktop.py
```

14. One-Sentence Summary

The project is shaped as a pipeline:

```
STEP geometry -> cleaning/neighbor features -> 10-feature LightGBM model -> dirty-face probabilities -> UI coloring
```